# anvil: Composable Code Transformations for R

*by Sebastian Fischer, Daniel Falbel, Thomas Kalinowski, Niko German, Martin Binder, and Bernd Bischl*

**Abstract** The **anvil** package offers a composable code transformation framework that allows users to express numerical algorithms efficiently in pure R. All code transformations are based on first tracing R code into a computational graph (an intermediate representation) via abstract evaluation. This allows combining jit-compilable operations with standard R code, the latter being "staged out" during compilation. Code transformations are implemented as functions that transform a computational graph into another computational graph. Except for the backends, the package is primarily written in R. Because of this and its modular design, **anvil** is easy to extend and contribute to, e.g. for adding new primitives or code transformations.

### 0.1 Introduction

All statistical and machine learning models fundamentally rely on some form of vector or tensor operations. Even though R as an interpreted language has performance limitations, relying on optimized matrix operations can often be fast enough for many use cases. However, there are cases where the performance is not sufficient and one needs to resort to compiled languages such as C++. Furthermore, one might additionally want to have access to GPU acceleration or automatic differentiation (AD) for gradient-based optimization.

To address this, we introduce **anvil**, which is a composable code transformation framework for the R language. Its primary goal is to offer a set of primitive operations that can be jit-compiled, differentiated, and executed on different hardware backends. It therefore allows users to implement numerical algorithms efficiently in pure R. Two different execution backends are supported: an OpenXLA ("Open Accelerated Linear Algebra") backend and an experimental Fortran backend. The OpenXLA project is a compiler ecosystem used across different array computing frameworks. Part of this project are the StableHLO language, which is a strongly typed language for representing array programs, and the PJRT (Pluggable Jit RunTime) library, which provides a common runtime for executing compiled programs. Components of **anvil** are the **stablehlo** and **pjrt** packages. The former allows users to easily create StableHLO programs in R, while the latter interfaces the PJRT C library and handles compilation, data allocation, and execution. Besides the OpenXLA backend, there is also an alternative, experimental backend that transpiles R to Fortran code via the **quickr** package. Because the **anvil** package is primarily written in R, it is easy to extend and contribute to.

The closest related work is the JAX python library, from which various design elements are inspired or adopted (Bradbury et al., 2018). Primarily, we are using a similar functional design and handle jit compilation via OpenXLA. Also other design elements such as the handling of nested data structures or the type promotion rules are adopted from JAX. We also want to highlight the Autodidax tutorial, which describes the core ideas underpinning JAX (todo: cite), as well as microjax, which is a minimalistic Python library demonstrating JAX's code transformation engine (todo: cite). For the design of the **stablehlo** package and the tracing implementation in **anvil**, we have followed the design from **mlr3torch**, which offers a similar graph-building interface for creating neural networks. The API functions for the tensor operations are partly adopted from NumPy, which is a popular library for numerical computing in Python (Harris et al., 2020). For the design of the **pjrt** package, we have partly borrowed from the gopjrt library (Pfeifer, 2026) which offers a go interface to PJRT.

The OpenXLA project (OpenXLA Team, 2026) is an open-source compiler ecosystem for machine learning, originally developed at Google as part of TensorFlow and later released

as a standalone project. One of its goals is to decouple numerical computing frameworks (the frontends) from hardware-specific implementations (the backends) by defining two standardized interfaces. One of these is the StableHLO language (StableHLO Team, 2026), a portable intermediate representation for tensor programs. StableHLO is built on MLIR (Multi-Level Intermediate Representation), which is itself part of the LLVM compiler infrastructure. MLIR provides a framework for defining and transforming intermediate representations at multiple levels of abstraction that are transformed into each other via passes that e.g. optimize the program or lower the abstraction level. The second interface is PJRT (Pluggable JIT RunTime), which defines a common API for the essential operations of a JIT-compiled tensor computing framework such as buffer management, compilation, and execution. PJRT follows a plugin architecture, where each vendor can provide a shared library that implements the PJRT interface for their specific hardware. This means that any framework that emits StableHLO and uses PJRT for execution can automatically target any hardware for which a PJRT plugin exists, including CPUs, NVIDIA GPUs, and Google TPUs. Currently, the main compiler consuming StableHLO programs is the XLA compiler, but alternatives such as the IREE compiler (The IREE Authors, 2019) also exist.

The main motivation behind developing **anvil** is to go beyond the limitations of the **torch** package (Falbel and Luraschi, 2025), which is a deep learning framework for the R language. It is based on LibTorch, which is the C++ library underpinning PyTorch (Paszke et al., 2019). The **torch** package offers a collection of highly optimized tensor operations, as well as various modules for building and training neural networks. As such, it also has support for backward-mode automatic differentiation and can be run on different hardware backends, including CPU and GPU. Thus, its general capabilities are similar to those of **anvil**, although it is more mature and contains more features. However, **torch** is primarily designed around so-called "eager" execution. For example, the forward function of a neural network is simply an R function that repeatedly calls into LibTorch kernels; i.e., there is no compilation step. While this has advantages regarding usability and expressiveness, it comes with performance implications. For large neural networks that are in the primary focus of modern deep learning toolboxes, these performance implications might be less problematic, as the runtime is dominated by individually expensive operations such as the attention mechanism. However, when models are smaller—as is not unusual for statistical models—JIT compilation can be especially attractive. While torch offers a way to JIT-compile code via tracing code and converting it to TorchScript, this feature is in maintenance mode. Furthermore, it is primarily designed for deployment and has, for example, no support for automatic differentiation.

Besides **torch**, other R packages exist that offer GPU acceleration and automatic differentiation. These are **tensorflow** and **keras3**, two deep learning frameworks that interface the TensorFlow Python runtime via **reticulate**. It is also possible to use Python's JAX library via **reticulate** (**?**). All of these packages also rely on OpenXLA for execution. While they provide access to more mature frameworks than **anvil**, they require managing the Python dependency, as well as type conversions between R and Python. Furthermore, with **anvil** our goal is to develop a tensor computing library that is and will be developed primarily with the needs of the R community in mind.

Beyond those popular deep learning libraries, other R packages exist that offer hardware acceleration, automatic differentiation, jit compilation, or a combination of these. We here primarily focus on those packages that are either available from standard repositories such as CRAN or Bioconductor, or which seem to be ready to be used.

A recent package that enables JIT compilation in R is the **quickr** package, which transpiles R code into Fortran code and then compiles it (Kalinowski, 2026). There are various differences between **anvil** and **quickr**. First, **quickr** has no support for automatic differentiation and currently only runs on CPU. Besides, **quickr** does not establish a new API, but instead aims to be a drop-in replacement for a set of R functions. This works because it is implemented via a static analysis approach, so the package does not need control over the functions that can be compiled, as is the case for the tracing approach followed in **anvil**. This also means that **quickr** does not allow mixing "non-compilable" R code with compilable

code, as is possible in **anvil**, where the "non-compilable" code is staged out during tracing to the intermediate representation. The tracing approach is more flexible, but also requires the user to be more careful about the code that they are writing. There is also the ast2ast package, which translates a subset of R code into C++ (Konrad Krämer, 2026). It is similar in spirit to **quickr**, but also supports forward and backward-mode automatic differentiation. Then, there are **odin** and **rxode2**, both of which focus on differential equations and compile a small subset of R code via translation to C, similar to **quickr**. However, they do not support GPU acceleration and their set of supported operations is more limited than **anvil**'s. There is another category of packages that offer special DSLs for statistical models which are compiled. For example, **brms** (Bürkner, 2017) allows users to define Bayesian models via the familiar formula interface and translates them into Stan models which are compiled (Carpenter et al., 2017). Stan itself is a probabilistic programming language and also has GPU support via OpenCL (Munshi, 2009). It is also possible to directly use Stan in R, e.g. via the **rstan** package, but this requires writing the program itself in Stan. There also exists **nimble**, which is primarily intended for fitting hierarchical statistical models and works by generating C++ code and compiling it. It also supports compiling custom R functions by generating C++ code, as well as forward and backward-mode automatic differentiation via CppAD (Bell, 2012). However, there is no support for GPU acceleration. Then, there is also the **compiler** package of the R language itself (R Core Team, 2025). It is used by the R interpreter to compile arbitrary R code into byte-code that is executed by the R virtual machine. However, because it has to handle arbitrary R code, the possible optimizations are limited due to R's design (Morandat et al., 2012). This generally highlights the trade-off between expressiveness and performance. In **anvil**, we are focusing on a rather narrow set of numerical operations, which thus allows for aggressive optimizations and highly efficient execution.

While various R packages offer GPU acceleration for specific operations (e.g., **xgboost**), generic GPU-accelerated array computing frameworks beyond the deep learning libraries from earlier are rare. One noteworthy mention is **OpenCL**, which offers a low-level interface to OpenCL.

Besides the packages already listed above, other R libraries exist with automatic differentiation capabilities. For example, forward mode automatic differentiation via dual numbers is supported by the packages **nabla**, **salad**, **dual**, and **madness** (Towell, 2026, Perdry (2024), Sartore (2025), Pav (2023)). None of these packages have JIT compilation capabilities or support GPU execution.

There is also the **RTMB** package which allows for forward and backward-mode automatic differentiation and interfaces the TMB C++ library ((Kristensen et al., 2016)).

Despite many packages offering some subset of **anvil**'s features, none of the native R packages simultaneously offer GPU acceleration, JIT compilation, and backward-mode automatic differentiation.

The structure of the paper is as follows: First, we will briefly review some important background information, including the OpenXLA compiler, the **quickr**, and some basics on automatic differentiation. In the subsequent part, we will describe the main design of the **anvil** package. Then, we will demonstrate the package's abilities, using logistic regression as a simple example. Afterwards, we will show how it is possible to extend the package. Then, we will include some small CPU runtime benchmarks to illustrate the performance of the package. Finally, we will conclude by discussing current limitations and directions for future work.

## 0.2 Background

### The OpenXLA Compiler Ecosystem

The OpenXLA compiler ecosystem consists of various individual components, but we will only cover the StableHLO intermediate language and the PJRT library which are used directly in **anvil**. One of the main goals of the OpenXLA project is to establish

standardized interfaces between numerical computing frameworks such as JAX or anvil and their underlying execution engines. This is achieved via the following two components:

1. The "Pluggable Jit RunTime" (PJRT) which defines a common interface for execution engines.
2. The StableHLO language, which is the intermediate representation produced by the frontend.

If a numerical computing framework produces StableHLO programs and handles the execution via PJRT, it can be run on any hardware for which a PJRT plugin exists. This turns an $n \times m$ problem ($n$ frontends need to implement their execution for $m$ different hardware backends) into a $n + m$ problem.

**StableHLO** An example for the StableHLO intermediate representation is given below. The main program takes in two `f32` tensors of shape `2x2`, multiplies them, and returns the result.

```
func @main(%arg0: tensor<2x2xf32>, %arg1: tensor<2x2xf32>) -> tensor<2x2xf32> {
 %0 = "stablehlo.multiply"(%arg0, %arg1) : (tensor<2x2xf32>, tensor<2x2xf32>) -> tensor<2x2xf32>
  return %0 : tensor<2x2xf32>
}
```

It is expressed in MLIR (Multi-Level Intermediate Representation). MLIR is part of the LLVM compiler infrastructure and provides a framework to define intermediate representations at multiple levels, also called dialects, that are transformed into each other via so-called passes. Such passes can e.g. perform optimizations or lower the abstraction level. It is in single static assignment (SSA) form, meaning that each variable is assigned to exactly once, and is fully functional. For a description of the language, see the StableHLO specification, which also contains a description of the around 100 supported primitive operations. In order to use the StableHLO language in **anvil**, it needs to be possible to translate R code into StableHLO programs. For this purpose, we have created the **stablehlo** package. The package essentially consists of four parts:

1. An S3 class representation of the StableHLO language, which closely follows the specification.
2. A `repr()` generic that converts the class representation into a string representation that can be compiled by the XLA compiler.
3. A set of type inference functions for the primitive operations.
4. A builder API for easily building up StableHLO programs.

The package is implemented fully in R, instead of interfacing the C++ implementation, which makes it easy to install for end users. However, the **stablehlo** package is only a partial implementation of the language and does not support all features. For example, there is no support for complex numbers, a few primitive operations are missing, and there is currently no support for shape dynamism (unknown dimensions).

The example below illustrates how to use the builder API to re-create the StableHLO program from above.

```
library(stablehlo)
func <- local_func("main")
func

#> func.func @main () ->  {
#>
#> }
```

```
x <- hlo_input("x", "f32", shape = c(2, 2), func = func)
x

#> Variable %x in:
#> func.func @main (%x: tensor<2x2xf32>) ->  {
#>
#> }

y <- hlo_input("y", "f32", shape = c(2, 2), func = func)
y

#> Variable %y in:
#> func.func @main (%x: tensor<2x2xf32>, %y: tensor<2x2xf32>) ->  {
#>
#> }

z <- hlo_multiply(x, y)
z

#> Variable %0 in:
#> func.func @main (%x: tensor<2x2xf32>, %y: tensor<2x2xf32>) ->  {
#> %0 = "stablehlo.multiply" (%x, %y): (tensor<2x2xf32>, tensor<2x2xf32>) -> (tensor<2x2xf32>)
#> }

f <- hlo_return(z)
f

#> func.func @main (%x: tensor<2x2xf32>, %y: tensor<2x2xf32>) -> tensor<2x2xf32> {
#> %0 = "stablehlo.multiply" (%x, %y): (tensor<2x2xf32>, tensor<2x2xf32>) -> (tensor<2x2xf32>)
#> "func.return"(%0): (tensor<2x2xf32>) -> ()
#> }

hlo_string <- repr(f)
```

What happens in the background is that first, a new `Func` object is created with the name "main", which has reference semantics. Then, `hlo_input()` creates two `FuncInput` objects with the names "x" and "y" and specifies their type. These are added to the `$inputs` field of the built-up `Func` object. The return values are `FuncValue` objects, which represent variables in the underlying StableHLO program that is being built up. Objects of type `FuncValue` represent a specific value in a StableHLO program and are the core data structure underpinning the builder API. Note that the R variable name `x` that is bound to the `FuncValue` does not need to coincide with the identifier "x" of the value in the StableHLO program. Further note that the `func` object here is passed explicitly, but this would not be necessary as the default is to use the function that was last created. When `x` and `y` are multiplied via `hlo_multiply()`, the corresponding type inference function `stablehlo::infer_types_multiply()` is called, which checks that the operation is valid for the given input types, computes the output type of the operation, registers the operation in the body of the underlying StableHLO program, and then returns a `FuncValue` object for the result. Then, `hlo_return()` sets the return values in the underlying `Func` object and returns the `Func` object itself, marking the end of the program creation.

Finally, the `repr()` generic is called on the `Func` object to get the string representation of the program. This is implemented by recursively calling the `repr()` generic on its sub-components. Note that the printer for `Func` and `FuncValue` objects uses the `repr()` generic internally to print the sub-components.

While some details were omitted in the above example, this shows the core ability of the **stablehlo** package to easily create StableHLO programs in R. For some more details, see the StableHLO specification or this vignette.

**PJRT** The pjrt package is the interface to the PJRT C++ library. The PJRT library itself is header-only and only defines the interface for the underlying execution engine. A specific implementation of the interface (e.g. for CPU or NVIDIA GPU) is accessed by downloading prebuilt shared libraries and loading them dynamically. Currently, these prebuilt PJRT plugins are obtained from the zig machine learning framework (zml). Core components of the PJRT interface are buffer allocation, program compilation, and program execution, but many other API endpoints exist. Just like for the stablehlo package, we only show the core functionality of pjrt here and refer to the corresponding vignette for more details.

As a first step, we will compile the StableHLO program from above. For this, the R `character(1)` first needs to be converted into a `PJRTProgram` object.

```
library(pjrt)
program <- pjrt_program(src = hlo_string, format = "mlir")
```

Then, this `PJRTProgram` can be compiled into a `PJRTLoadedExecutable` object. This requires specifying a `PJRTClient`, as well as some optional compile options. The `PJRTClient` contains a `PJRTPlugin`, which is a dynamic library that implements the PJRT interface for a specific hardware backend. It can be specified via a string identifier such as "cpu" or "cuda" or we can pass a `PJRTPlugin` object directly. Here, we choose the latter for demonstration purposes.

```
client <- pjrt_client("cpu")
executable <- pjrt_compile(program, client = client)
executable

#> <PJRTLoadedExecutable>
```

In order to execute this program, `PJRTBuffer` objects need to be created that are multi-dimensional tensors. Below, we create two simple `f32` tensors of shape `2x2` and specify the device to be the CPU, which will use the buffer allocation implementation of the PJRT CPU plugin. For CPU, the distinction between device and client is not so important as there is usually only one CPU, so there is a one-to-one correspondence. When working with GPUs, there might be multiple GPUs available and the device represents one of them (by default the first one), and it matters on which GPU the buffer will be allocated. However, multi-GPU setups are not properly supported in pjrt yet, but this could be added in the future.

```
x <- pjrt_buffer(1:4, dtype = "f32", device = "cpu")
x

#> PJRTBuffer
#>  1
#>  2
#>  3
#>  4
#> [ CPUf32{4} ]

y <- pjrt_buffer(5:8, dtype = "f32", device = "cpu")
y

#> PJRTBuffer
#>  5
#>  6
#>  7
#>  8
#> [ CPUf32{4} ]
```

Various extractors exist for `PJRTBuffer` objects, for example:

```
shape(x)

#> [1] 4

device(x)

#> <CpuDevice(id=0)>
```

We can multiply the two buffers with each other by passing them to the `pjrt_execute()` function along with the executable.

```
out <- pjrt_execute(executable, x, y)
out

#> PJRTBuffer
#>   5 12
#>  21 32
#> [ CPUf32{2x2} ]
```

Conversion back to R is possible via the `as_array()` function.

```
as_array(out)

#>      [,1] [,2]
#> [1,]    5   12
#> [2,]   21   32
```

Because `PJRTBuffer` objects are external pointers, custom (de)serialization methods are needed. For this, we implement the safetensors format (Falbel, 2025).

```
library(safetensors)
tmp <- tempfile(fileext = ".safetensors")
safetensors::safe_save_file(list(x = out), tmp, framework = "pjrt")
reloaded <- safetensors::safe_load_file(tmp, framework = "pjrt")
reloaded$x

#> PJRTBuffer
#>   5 12
#>  21 32
#> [ CPUf32{2x2} ]
```

### quickr

The **quickr** package serves as a second, experimental, backend for **anvil** and was independently developed. It transpiles R code to Fortran code and compiles it. Below, we can recreate the StableHLO program from above. One semantic difference is that it uses data type `f64` instead of `f32` because **quickr** operates directly on R as opposed to **pjrt** which defines its own tensor type.

```
library(quickr)

multiply_r <- function(x, y) {
  declare(
    type(x = double(NA, NA)),
    type(y = double(NA, NA))
  )
```

```
  x * y
}
f <- quick(multiply_r)
f(
  matrix(c(1, 2, 3, 4), nrow = 2),
  matrix(c(5, 6, 7, 8), nrow = 2)
)

#>      [,1] [,2]
#> [1,]    5   21
#> [2,]   12   32
```

There are also various other differences with respect to the underlying implementation. Instead of offering a builder API like in **stablehlo**, **quickr** works via static analysis of the R code. Furthermore, the argument types of the inputs need to be specified via the `declare()` function. Here, `double(NA, NA)` means a 2-dimensional array with unknown dimensions, but concrete dimensions can also be specified. One advantage of **quickr** over **pjrt** and **stablehlo** is that the former does not necessitate recompilation for every new input type combination (which includes the shapes). A disadvantage is that **quickr** currently does not support GPU acceleration and also less data types.

### Automatic Differentiation

Besides JIT compilation, another core feature of **anvil** is its ability to automatically compute derivatives of R functions. Here, we briefly cover some central ideas behind AD to provide the background for understanding **anvil**'s design. For a comprehensive treatment, we refer the reader to Griewank and Walther (2008).

**Evaluation Procedures and the Chain Rule** AD exploits the fact that every computer program, no matter how complex, executes a sequence of elementary operations whose derivatives are known. Following Griewank and Walther (2008), consider a function $F : \mathbb{R}^n \to \mathbb{R}^m$ that is evaluated by a computer program. Any such evaluation can be decomposed into a sequence of elementary operations (called *elemental functions*) $\varphi_i$ such as addition, multiplication, sin, exp, etc. The evaluation produces a sequence of intermediate variables

$$v_i = \varphi_i(v_j)_{j \prec i}, \qquad i = 1, \ldots, l$$

where $v_{1-n}, \ldots, v_0$ are the input variables (i.e., $x$), $v_1, \ldots, v_l$ are the intermediate variables, and $v_{l-m+1}, \ldots, v_l$ are the output variables (i.e., $y$). Here, the notation $j \prec i$ indicates that $v_j$ is a direct argument of $\varphi_i$. This sequence of assignments is called an *evaluation procedure* and can be visualized as a directed acyclic graph (the *computational graph*), where each node corresponds to an intermediate variable and edges represent data dependencies.

Since each elemental function $\varphi_i$ is differentiable, the chain rule can be applied systematically to compute derivatives of $F$. The two main modes of AD differ in the *order* in which the chain rule is evaluated, leading to different computational trade-offs.

**Forward Mode** Forward-mode AD propagates derivatives alongside the function evaluation. Given a tangent (or direction) vector $\dot{x} \in \mathbb{R}^n$, it computes for each intermediate variable $v_i$ the corresponding tangent

$$\dot{v}_i = \sum_{j \prec i} \frac{\partial v_i}{\partial v_j} \dot{v}_j$$

proceeding forward through the evaluation procedure, from inputs to outputs. The final result $\dot{y} = F'(x)\dot{x}$ is a *Jacobian-vector product* (JVP). To obtain the full Jacobian $F'(x) \in \mathbb{R}^{m \times n}$,

one must perform $n$ forward sweeps, one for each standard basis vector $\dot{x} = e_j$. This makes forward mode efficient when $n$ is small relative to $m$.

**Reverse Mode** Reverse-mode AD (also called the *adjoint* mode) propagates sensitivities backward through the evaluation procedure. Given an adjoint vector $\bar{y} \in \mathbb{R}^m$, it computes for each intermediate variable $v_i$ the corresponding adjoint

$$\bar{v}_i = \sum_{k \succ i} \bar{v}_k \frac{\partial v_k}{\partial v_i}$$

proceeding in reverse order, from outputs to inputs, where $k \succ i$ denotes that $v_i$ is a direct argument of $\varphi_k$. The final result $\bar{x} = \bar{y}^\top F'(x)$ is a *vector-Jacobian product* (VJP).

The key advantage of reverse mode is that for a scalar-valued function ($m = 1$), setting $\bar{y} = 1$ yields the full gradient $\nabla F(x) \in \mathbb{R}^n$ in a *single* reverse sweep, regardless of $n$. This result is sometimes referred to as the "cheap gradient principle": the computational cost of evaluating the gradient is bounded by a small constant multiple (typically 2–5) of the cost of evaluating $F$ itself (Griewank and Walther (2008), Chapter 3). This makes reverse mode dramatically more efficient than forward mode for the common case of gradient-based optimization, where one differentiates a scalar loss with respect to potentially many parameters.

The trade-off is that reverse mode requires access to the intermediate values $v_i$ from the forward evaluation during the backward sweep, since they are needed to evaluate the local partial derivatives $\frac{\partial v_k}{\partial v_i}$. This introduces additional memory overhead that scales with the depth of the computation.

In **anvil**, we implement backward-mode AD as a graph-to-graph transformation, as described in the next section. Each primitive operation stores its own local VJP rule, and the transformation walks the computational graph in reverse topological order to accumulate gradients.

### 0.3 Package Design and Features

Having coverved the underlying basics, we will now cover the main design of the **anvil** frontend. The core idea of the package is centered around code transformations, an approach adopted from JAX (**?**). In general, there are three types of transformations:

1. Transforming R code into a computational graph.
2. Transforming a computational graph into another computational graph (e.g. for automatic differentiation).
3. Transforming a computational graph into an executable (either via **stablehlo** and **pjrt** or **quickr**).

We will now cover these three types of transformations in more detail. Then, we explain how these transformations are wrapped in the main user interface.

#### Code transformations

**Tracing R code into a computational graph** When it comes to translating code into other code, there are in principle two approaches:

1. Static analysis of the code.
2. Recording an *evaluation trace* of the code by abstractly evaluating it.

In **anvil**, we are following the latter approach, because it allows to seemlessly combine standard R code with jit-compilable operations. The function that takes in an R function and returns a computational graph is called `trace_fn()`. In order for this to be useful, the

function needs to call into **anvil** primitive functions. These primitive functions are similar to the stableHLO operations like `hlo_add` from earlier.

In the **anvil** package these are called `nvl_<op>` functions. Howveer, they are not exported directly, but in thin wrappers like `nv_add`, which perform some convenience transformations (like type casting) that make them more ergonomic to use. These thin wrappers call into the primitive functions, however. The primitive functions are essentially graph building functions, also similar to the `hlo_add` function from earlier. Below, we create an R function that takes in two values and multiplies them. Note that we could have also used the `*` infix operator, but we want to illustrate the use of the `nv_multiply` function.

```
library(anvil)
multiply_r <- function(x, y) {
  nv_mul(x, y)
}
```

We can now trace this function into a computational graph. At the beginning, this will create a `anvil::GraphDescriptor` that primitive functions can attach to to build up the graph. The types for the inputs are inferred from the example arguments. The `trace_fn` transformation uses these example inputs to obtain abstract `GraphBox` objects, which are passed around during the evaluation of the function. They essentially define the type (shape and data type), and uniquely identify a specific `GraphValue` within a specific `GraphDescriptor`. Note that `trace_fn` also takes in non-tensor values, which are evaluated like normal R code.

```
graph <- trace_fn(multiply_r, list(x = nv_scalar(1, "f32"), y = nv_scalar(5, "f32")))
graph
```

```
#> <AnvilGraph>
#>   Inputs:
#>     %x1: f32[]
#>     %x2: f32[]
#>   Body:
#>     %1: f32[] = mul(%x1, %x2)
#>   Outputs:
#>     %1: f32[]
```

Once the evaluation of `multiply_r` with the special `GraphBox` objects is complete, the `GraphDescriptor` is converted into a `Graph` object. The only difference between them is that `GraphDescriptor` does some book-keeping during program creation which is not needed anymore afterwards.

From the printed output of the graph we can see that a graph has:

1. Inputs: `GraphValues` that are the inputs to the graph.
2. Constants: `GraphValues` that are constants in the graph.
3. Body: A list of `PrimitiveCall` objects, which represent the primitive operations that make up the graph.
4. Outputs: `GraphValues` that are the outputs of the graph.
5. In-tree and Out-tree: Because `Graphs` only take in tensor values, but we want to support nested data strctures, the in-tree and out-tree define how to (un-)flatten the inputs and outputs of the graph.

STATIC SHAPES!!!

**Transforming a Graph into another Graph** Once our program no longer consists arbitrary R code, but is in a standardized form, we can transform the program by creating another `Graph` object. Currently, the only example for such a type of transformation is the gradient transformation, which is implemented in the `transform_gradient()` function. It takes in a `Graph` and the names of the arguments to compute the gradient with respect to.

```
gradient_graph <- transform_gradient(graph, wrt = c("x", "y"))
```

The **anvil** package does not put any restrictions on *how* such a transformation is implemented. Usually, however, such a transformation needs to store information for how to transform each primitiveF. For this reason, the `anvil::Primitive` objects have a `@rules` field that can be populated. For example, the backward rule for the addition primitive is implemented as follows:

```
prim("add")[["backward"]]

#> function (inputs, outputs, grads, .required)
#> {
#>     grad <- grads[[1L]]
#>     list(if (.required[[1L]]) grad, if (.required[[2L]]) grad)
#> }
#> <bytecode: 0x140ed3190>
#> <environment: namespace:anvil>
```

The transformation function `transform_gradient` basically walks the graph backwards and applies the backward rules to each primitive, producing a `Graph` that contains the forward and backward pass.

**Automatic Differentiation** The `transform_gradient()` function implements backward-mode automatic differentiation (also known as reverse-mode AD). Given a `Graph` representing a function $f : \mathbb{R}^n \to \mathbb{R}$, it produces a new `Graph` that computes both the original output and the gradients of the output with respect to specified inputs.

More concretely, consider a computational graph as a sequence of primitive operations $v_k = g_k(v_{\text{pa}(k)})$, where $v_{\text{pa}(k)}$ denotes the parent values (inputs) of operation $k$. The goal is to compute $\bar{v}_i = \frac{\partial f}{\partial v_i}$ for each intermediate value $v_i$. Starting from $\bar{f} = 1$, the backward pass walks the graph in reverse topological order and accumulates gradients via the chain rule:

$$\bar{v}_i = \sum_{k: i \in \text{pa}(k)} \bar{v}_k \cdot \frac{\partial g_k}{\partial v_i}$$

Each primitive's backward rule computes the local partial derivatives $\frac{\partial g_k}{\partial v_i}$ and multiplies them with the incoming gradient $\bar{v}_k$. This is the standard vector-Jacobian product (VJP) formulation of reverse-mode AD, which avoids materializing the full Jacobian matrices of intermediate operations. Instead, each backward rule directly computes the product of the incoming gradient vector with the local Jacobian, which is typically much cheaper and can often be expressed in closed form.

In the implementation, the transformation walks the graph in reverse order. For each `PrimitiveCall`, it looks up the corresponding backward rule stored in the primitive's `@rules[["backward"]]` field. This backward rule receives the original inputs and outputs of the primitive call (from the forward pass) as well as the incoming gradient $\bar{v}_k$, and returns the gradient contributions for each input. These contributions are accumulated across all uses of a value to obtain the total gradient $\bar{v}_i$.

Because the backward rules are themselves expressed in terms of primitive operations (i.e., they call `nvl_*` functions), the gradient graph is built using the same tracing mechanism as the forward pass. This means that the gradient computation is itself a `Graph` that can be further transformed or compiled — for example, one could in principle compute higher-order derivatives by applying `transform_gradient()` again to the gradient graph.

An important design choice is that the backward rules are stored per-primitive in the `@rules` field, rather than being hard-coded in the transformation. This makes it straightforward to add differentiation support for new primitives (see Section 2.0.5) and keeps the transformation logic generic.

**Compiling a Graph** Whenever we are done transforming out `Graph` and are ready to execute it, we need to lower it. We can convert a `Graph` into a stablehlo program via the `stablehlo()` function. The output of this is a list containing:

1. The StableHLO program.
2. The constant buffers in the `Graph`.

```
out <- stablehlo(graph)
repr(out[[1]])
```

```
#> [1] "func.func @main (%0: tensor<f32>, %1: tensor<f32>) -> tensor<f32> {\n%2 = \"stablehlo.multiply
```

```
out[[2]]
```

```
#> list()
```

While stablehlo allows to embed constants in the program, this can considerably increase the program size (think of big tensors) and slow down compile times. For this reason, non-scalar (TODO) constants are not inlined into the stableHLO executable, but added as inputs to the program and need to be provided when executing the program.

Next, we will describe how these transformations are wrapped in the main user interface.

### Main User Interface

The main user interface is centered around `jit()` and transformations functions like `gradient()` or `value_and_gradient()`. Furthermore, we can create tensors, which we will cover first.

**The `AnvilTensor`** In order to create data for our programs, we can use the `nv_tensor()`, `nv_scalar()` and `nv_empty()` functions. These essentially wrap the `pjrt_buffer()` and `pjrt_scalar()` functions from the **pjrt** package.

```
x <- nv_scalar(1, "f32")
x
```

```
#> AnvilTensor
#>  1
#> [ CPUf32{} ]
```

```
y <- nv_tensor(1:4, "f32")
y
```

```
#> AnvilTensor
#>  1
#>  2
#>  3
#>  4
#> [ CPUf32{4} ]
```

Just like the `PJRTBuffer` objects, `AnvilTensor` objects can be serialized to a string and deserialized using the **safetensors** R package.

**`jit()`** `jit()` is the main function that takes in an R function and returns a JIT-compiled function Below, we apply it to a simple R function that prints a message and then multiplies two values. The print statement is included for didactive purposes and will become clear later.

```
multiply_print_r <- function(x, y) {
  print("Hello useR!")
  nv_mul(x, x)
}
multiply_jit <- jit(multiply_print_r)
```

The output function is a regular R function, which contains the "recipe" for transforming the R function into a `Graph`, then lowering into a StableHLO program and finally executing it.

```
multiply_jit(x, y)

#> [1] "Hello useR!"

#> AnvilTensor
#>  1
#> [ CPUf32{} ]
```

Of course, compiling the function every time the function is called would be very inefficient. Therefore, a cache is maintained that stores the executable (and constant values) for each unique combination of input types.

If we execute it again, we get the same output, but the print statement is not shown.

```
multiply_jit(x, y)

#> AnvilTensor
#>  1
#> [ CPUf32{} ]
```

If we apply it again to different input values (of the same types), we get a cache hit and directly execute the cached executable.

```
multiply_jit(nv_scalar(2, "f32"), nv_scalar(3, "f32"))

#> [1] "Hello useR!"

#> AnvilTensor
#>  4
#> [ CPUf32{} ]
```

Because the compiled executable contains only the tensor operation, the print statement is not shown. If we apply the function to values of different types, we get a cache miss and the function is recompiled. This will show the print message again.

```
multiply_jit(nv_scalar(1, "i32"), nv_scalar(5, "i32"))

#> [1] "Hello useR!"

#> AnvilTensor
#>  1
#> [ CPUi32{} ]
```

We can also JIT-compile functions that take in lists of tensors and output lists of tensors.

```
multiply_jit_nested <- jit(function(lst) {
  list(nv_mul(lst[[1]], lst[[2]]))
})
multiply_jit_nested(list(x, y))

#> [[1]]
#> AnvilTensor
#>  1
#>  2
#>  3
#>  4
#> [ CPUf32{4} ]
```

This will use the `in_tree` and `out_tree` fields of the `Graph` to flatten and unflatten the list of tensors.

The functions we JIT-compile can also take in so called "static" arguemnts, that can be standard R values. Below, we define a function that either multiplies or adds two values, depending on the value of the `op` argument. We need to mark the `op` argument as static.

```
add_or_multiply <- jit(function(x, y, op) {
  if (op == "add") {
    nv_add(x, y)
  } else {
    nv_mul(x, y)
  }
}, static = "op")
```

The difference between static and non-static arguments is that for the former the function is recompiled for ever unique input type combination, while for the latter it needs to be recompiled for every unique value of the static argument.

```
add_or_multiply(x, y, "add")

#> AnvilTensor
#>  2
#>  3
#>  4
#>  5
#> [ CPUf32{4} ]

add_or_multiply(x, y, "multiply")

#> AnvilTensor
#>  1
#>  2
#>  3
#>  4
#> [ CPUf32{4} ]
```

**Transformations like `gradient()`** Just like `jit()`, the `gradient()` function takes in a function and outputs a function that contains the recipe for transforming the input function into a `Graph` representing the gradient of the input function.

```
add_fn <- function(x, y) {
  nv_add(x, y)
}
grad <- gradient(add_fn, wrt = c("x", "y"))
```

In order to be able to execute the gradient function, we need to wrap it in a `jit()` call. We can also jit a function that contains "standard" anvil operations and calls into the gradient function.

```
x <- nv_scalar(1, "f32")
y <- nv_scalar(5, "f32")
f <- function(x, y) {
  z <- nv_add(x, y)
  grads <- grad(x, z)
  grads
}
```

The way this works is the following: When tracing the `val_and_grad` function, a `GraphDescriptor` is first initialized. When the `add_or_multiply` function is called, it populates the `GraphDescriptor` with the inputs `x` and `y` and adds the `PrimitiveCall` to the body representing the multiplication of `x` and `y`. When we then call into the `grad` function, we create another `GraphDescriptor`, because we need a graph-representation of the of the `add_or_multiply` function so we can transform it into its gradient. After this is done, the resulting gradient graph is inlined into the parent `GraphDescriptor`.

We can see the output of this below:

```
graph <- trace_fn(f, list(x = x, y = y))
graph
```

```
#> <AnvilGraph>
#>   Inputs:
#>     %x1: f32[]
#>     %x2: f32[]
#>   Constants:
#>     %c1: f32[]
#>   Body:
#>     %1: f32[] = add(%x1, %x2)
#>     %2: f32[] = add(%x1, %1)
#>   Outputs:
#>     %c1: f32[]
#>     %c1: f32[]
```

Because this makes `f` just another graph-building function, we can wrap it in a `jit()` call and execute it.

```
f_jit <- jit(f)
f_jit(x, y)
```

```
#> $x
#> AnvilTensor
#>  1
#> [ CPUf32{} ]
#>
#> $y
#> AnvilTensor
#>  1
#> [ CPUf32{} ]
```

**Debugging** One of the main disadvantages of the JIT-compilation approach is that it makes the program harder to debug, because the function cannot simply be executed line-by-line. For this reason, we offer a debugging mode that allows to execute a function without wrapping it in a `jit()` call. There, the function will be executed line-by-line. However, this will mean that we don't execute the real function, but only perform the abstract evaluation, meaning you can also see the types of the intermediate values.

```
multiply_print_r(x, y)

#> [1] "Hello useR!"

#> f32{}
```

It is also possible to print intermediate tensors during program execution via `nv_print()`.

```
multiply_print_jit <- jit(function(x, y) {
  out <- nv_mul(x, y)
  nv_print(out)
  return(out)
})
multiply_print_jit(x, y)

#> AnvilTensor
#>  5
#> [ f32{} ]

#> AnvilTensor
#>  5
#> [ CPUf32{} ]
```

The difference between `nv_print` and a standard `print()` call is that the former will be embedded in the StableHLO program and executed by the backend, while the latter will be executed during the tracing. The `nv_print()` function is implemeted as a StableHLO custom call, c.f. the extending section.

Finally, to aid debugging, we try to provide good error messages when applying graph-building functions to the wrong types of inputs.

### Static Shapes

While the JIT-compilation approach is very powerful, it has some limitations. One major restriction is that the shapes in the StableHLO programs are static and need to be known at compile time. This means that the approach does not do well when the input shapes are varying strongly. In principle, the StableHLO language does support dynamic shapes. However, the XLA compiler which we are using requires all shapes to be known at compile time.

In order to lift this restriction, it would necessary to implement a different backend. One possible approach is to use the IREE compiler (cite, link), which also consumes StableHLO programs and does support dynamic shapes. Another approach might be to lower the `Graph` into a different language, e.g. using the **quickr** or by transpiling to Mojo. An advantage of the static shapes is that it allows for more aggressive optimizations by the compiler.

### The

## 0.4 Examples

Despite **anvil** being still in a rather early stage and there are many features we would like to add, it can already be used to accelerate real-world problems. We demonstrate this with

a simple logistic regression example that showcases `jit()`, `gradient()`, and `nv_while()` working together.

**Logistic Regression via Gradient Descent**

To illustrate how `jit()` and `gradient()` work together, we fit a logistic regression model via gradient descent using the Titanic survival dataset from base R. A more detailed version of this example is available as a package vignette.

We first prepare the data by expanding the contingency table into a design matrix and converting it to `AnvilTensors`.

```
set.seed(42)
titanic_df <- as.data.frame(Titanic)
titanic <- titanic_df[rep(seq_len(nrow(titanic_df)), titanic_df$Freq), 1:4]
X <- scale(model.matrix(~ Class + Sex + Age, data = titanic)[, -1])
y <- as.integer(titanic$Survived == "Yes")
n <- nrow(X); p <- ncol(X)

X_tensor <- nv_tensor(X, dtype = "f32")
y_tensor <- nv_tensor(y, dtype = "f32", shape = c(n, 1L))
```

The model predicts survival probabilities and evaluates them with binary cross-entropy loss:

```
model_loss <- function(X, y, beta, alpha) {
  probs <- nv_logistic(X %*% beta + alpha)
  eps <- 1e-7
  probs <- nv_clamp(eps, probs, 1 - eps)
  loss <- -(y * log(probs) + (1 - y) * log(1 - probs))
  mean(loss)
}
```

We obtain the gradient of the loss with respect to the parameters using `gradient()`:

```
model_loss_grad <- gradient(model_loss, wrt = c("beta", "alpha"))
```

The training loop is implemented using `nv_while()`, which keeps the entire loop within a single compiled function. This avoids the overhead of repeatedly calling into the compiled function from R.

```
fit_logreg <- jit(function(X, y, beta, alpha, n_epochs, lr) {
  output <- nv_while(
    list(beta = beta, alpha = alpha, epoch = nv_scalar(0L)),
    \(beta, alpha, epoch) epoch < n_epochs,
    \(beta, alpha, epoch) {
      grads <- model_loss_grad(X, y, beta, alpha)
      list(
        beta = beta - lr * grads$beta,
        alpha = alpha - lr * grads$alpha,
        epoch = epoch + 1L
      )
    }
  )
  list(beta = output$beta, alpha = output$alpha)
})
```

We initialize the parameters and train the model:

```
beta_init <- nv_tensor(rnorm(p), dtype = "f32", shape = c(p, 1L))
alpha_init <- nv_scalar(0, dtype = "f32")

result <- fit_logreg(
  X_tensor, y_tensor, beta_init, alpha_init,
  nv_scalar(50000L), nv_scalar(0.1)
)
```

To verify correctness, we compare the estimated coefficients with R's built-in `glm()`:

```
#>     Parameter      anvil        glm
#> 1 (Intercept) -0.8538058 -0.8538077
#> 2    Class2nd -0.3418888 -0.3418906
#> 3    Class3rd -0.8299900 -0.8299946
#> 4   ClassCrew -0.4206345 -0.4206313
#> 5   SexFemale  0.9919736  0.9919786
#> 6    AgeAdult -0.2303528 -0.2303616
```

The estimates closely match, confirming that the gradient descent implementation converges to the same solution as `glm()`.

Further examples, including TODO, are available as package vignettes (TODO).

### 0.5 Extending

One of the key design goals of **anvil** is to be extendable. It is possible to add new primitive operations, transformations, and even backends.

#### New Primitive Operations

The simplest way to extend **anvil** is to add new primitive operations. This consists of defining a `Primitive` object, implementing the transformation rules, and creating a graph-builder function for the primitive. Below, we show how to create the (already existing) addition primitive from scratch. We start by creating the `Primitive` object that holds the transformation rules.

```
p_mul_custom <- AnvilPrimitive("mul_custom")
```

Next, we create the graph-builder function for the primitive. It needs to define the abstract evaluation rule (the `infer_fn` argument) that describes how to convert the abstract inputs into abstract outputs. Then, it appends a `PrimitiveCall` object to the `GraphDescriptor` that represents the primitive call via `graph_desc_add()`. because `graph_desc_add` returns a list of values, we unbox it and return the first element.

```
nvl_mul_custom <- function(x, y) {
  graph_desc_add(p_mul_custom, list(x, y), infer_fn = infer_binary)[[1L]]
}
```

With this in place, we can already create `Graph` objects that use the new primitive.

However, we also want to be able to translate this to StableHLO and compute gradients.

We can define these rules by setting the `@rules` field of the `Primitive` object. We start with the stablehlo rule. It takes as inputs `FuncValue` objects and returns a list of `FuncValue` object. This is exactly what the inference functions from the **stablehlo** do, so we can just use it:

```
p_mul_custom[["stablehlo"]] <- function(lhs, rhs) {
  list(stablehlo::hlo_multiply(lhs, rhs))
}
```

Next, we define the backward rule. It is expected to be a function with the following arguments:

- `inputs`: The inputs to the primitive function during the forward pass.
- `outputs`: The outputs of the primitive function during the forward pass.
- `grads`: The gradients of the outputs of the primitive function during the backward pass.
- `required`: A logical vector indicating which inputs require a gradient.

The `inputs`, `outputs` and `grads` are list of `GraphBox` objects and the function should return a list of the same length as the inputs. The values of this list must either be a `GraphBox` containing the gradient of the input with respect to the output (respecting the chain rule), or `NULL` if the input does not require a gradient. The `required` argument is a logical vector indicating which inputs require a gradient.

```
p_mul_custom[["backward"]] <- function(inputs, outputs, grads, required) {
  grad <- grads[[1L]]
  list(
    if (required[[1L]]) nvl_mul(grad, rhs),
    if (required[[2L]]) nvl_mul(grad, lhs)
  )
}
```

From this implementation we can see that any primitive operations that can be expressed in terms of StableHLO operations can easily be added. While this is already quite powerful, there are cases where this is not enough. For this reason, it is in principle possible to implement custom calls that perform arbitrary operations defined in C++, possibly using CUDA. This is not super ergonomic yet, but this could be improved if it shows to be necessary.

#### New Transformations

Because the `Graph` representation is a simple data structure consisting of inputs, outputs, and a sequence of `PrimitiveCall` objects, it is straightforward to implement new graph-to-graph transformations. The `transform_gradient()` function, which implements backward-mode automatic differentiation, is the primary example of such a transformation. It walks the graph in reverse order and applies the backward rule stored in each primitive's `@rules[["backward"]]` field to accumulate gradients.

Implementing a new transformation follows the same pattern: iterate over the `PrimitiveCall` objects in the graph, look up a corresponding rule for each primitive, and use these rules to construct a new graph. For example, one could implement forward-mode automatic differentiation by walking the graph *forward* and propagating Jacobian-vector products, or implement a graph optimization pass that fuses or simplifies sequences of primitive calls. The key design decision is that the **anvil** package does not impose any constraints on how a transformation is implemented — it only provides the `Primitive` rule mechanism as a convenient way to store per-operation information.

#### Custom Backends

Adding a new backend amounts to implementing a new graph-to-executable transformation, analogous to how the XLA backend translates a `Graph` into a StableHLO program via the `stablehlo()` function and then compiles it via PJRT. For each primitive, a lowering

rule is stored in the `@rules` field (e.g., under `"stablehlo"` for the XLA backend). A new backend would define its own set of rules under a different name and provide a corresponding lowering function that walks the graph and applies these rules to produce the target representation.

As a concrete example, there is an experimental backend that compiles `Graph` objects to Fortran code via the **quickr** package, targeting CPU execution without requiring the XLA toolchain. This demonstrates that the same graph can be lowered to fundamentally different targets. However, the process of adding a new backend currently requires some familiarity with the package internals. Providing a more streamlined and well-documented interface for registering custom backends is an area we plan to improve.

### 0.6 Testing

Ensuring correctness of both the forward evaluation and the gradient computation is critical for a code transformation framework. In **anvil**, we bootstrap correctness by comparing against the **torch** package wherever possible.

For forward-pass testing, each primitive's JIT-compiled output is compared against the corresponding **torch** function on randomly generated inputs. For example, to test `nvl_add`, we generate random tensors, compute the result using both `jit(nvl_add)(x, y)` and `torch::torch_add(x, y)`, and assert that the outputs match up to numerical tolerance. This is done using helper functions like `expect_jit_torch_binary()` and `expect_jit_torch_unary()`, which handle the data generation and comparison.

For backward-pass testing, the same strategy is applied to gradient computation. The anvil gradient (obtained via `jit(gradient(f))(x)`) is compared against the gradient computed by **torch**'s autograd engine. For non-scalar functions, both sides are wrapped with a summation to produce a scalar loss before differentiating, ensuring a consistent comparison.

Because **torch** is not a hard dependency of the package, the torch-based tests are stored in `inst/extra-tests/` and are conditionally sourced only when **torch** is installed. This keeps the test suite lightweight for users who do not have **torch**, while still running the full comparison suite in CI. For primitives that have no direct **torch** equivalent (e.g., random number generation or bitwise operations), manual tests with hardcoded expected values are used instead.

### 0.7 Benchmarks

Runtime benchmarks comparing **anvil** on CPU and GPU are available at `https://r-xla.github.io/benchmarks/`.

### 0.8 Discussion

In this paper, we introduced the **anvil** package, which is a code transformation framework for the R language. Execution is handled via JIT compilation through either the OpenXLA compiler framework or by transpiling to Fortran via **quickr**. When using the OpenXLA backend, it is possible to make use of accelerators such as GPUs. Furthermore, the package supports first-order automatic differentiation.

The biggest limitations of the current approach are twofold:

a) Programs need to have static shapes, which means that operations such as `unique()` are currently not supported.
b) For the OpenXLA backend, programs need to be re-compiled for each new unique input type combination.

TODO: Tracing vs. Static analysis.

Future work will aim at lifting the static shape and re-compilation restrictions. One solution will be to improve the support for the Fortran backend, which – as opposed to XLA

– does support shape dynamism. To do this properly, the intermediate `AnvilGraph` needs to be extended to support shape dynamism, which it currently does not. Extending the Fortran backend to cover more primitives also means that the second major restriction can be lifted, at least when executing on CPUs.

Apart from these steps, there also also many other next steps that would improve useability and make it more generally useful. Another important improvement is to decouple the intermediate representation (`AnvilGraph`) from StableHLO more thoroughly. Currently, the IR is tied too closely to StableHLO, which limits the ability to target alternative backends and to represent higher-level optimizations that do not map directly to StableHLO concepts. The simplest one is to extend the set of primitive operations supported in both **anvil** and **stablehlo**. Then, more useful API functions can be added, e.g. focusing on linear algebra operations. Also, other code transformations such as higher-order derivatives, forward-mode automatic differentiation or improved vectorization transformations could be added.

Finally, one package that might be useful is to build a small modeling library on top of **anvil**, which is aimed at optimizing models via gradient-based optimization methods, e.g. for training deep learning models.

- JET.
- Improving the intermediate representation.
- Using {anvil} to implement common algorithms.

### 0.9 Acknowledgments

Sebastian Fischer is supported by the Deutsche Forschungsgemeinschaft (DFG, German Research Foundation) – 460135501 (NFDI project MaRDI).

### 0.10 Appendix

#### AI Usage

AI coding agents were used for the development of the software. However, all AI-generated code was reviewed by the authors.

## References

B. M. Bell. Cppad: a package for c++ algorithmic differentiation. *Computational Infrastructure for Operations Research*, 57(10):3, 2012. [p3]

J. Bradbury, R. Frostig, P. Hawkins, M. J. Johnson, C. Leary, D. Maclaurin, G. Necula, A. Paszke, J. VanderPlas, S. Wanderman-Milne, and Q. Zhang. Jax: Composable transformations of python+numpy programs. Github repository. URL https://github.com/jax-ml/jax, 2018. [p1]

P.-C. Bürkner. brms: An r package for bayesian multilevel models using stan. *Journal of statistical software*, 80:1–28, 2017. [p3]

B. Carpenter, A. Gelman, M. D. Hoffman, D. Lee, B. Goodrich, M. Betancourt, M. Brubaker, J. Guo, P. Li, and A. Riddell. Stan: A probabilistic programming language. *Journal of statistical software*, 76:1–32, 2017. [p3]

D. Falbel. *safetensors: Safetensors File Format*, 2025. URL https://CRAN.R-project.org/package=safetensors. R package version 0.2.0. [p7]

D. Falbel and J. Luraschi. *torch: Tensors and Neural Networks with 'GPU' Acceleration*, 2025. URL https://CRAN.R-project.org/package=torch. R package version 0.16.3. [p2]

A. Griewank and A. Walther. *Evaluating derivatives: principles and techniques of algorithmic differentiation*. SIAM, 2nd edition, 2008. doi: 10.1137/1.9780898717761. [p8, 9]

C. R. Harris, K. J. Millman, S. J. Van Der Walt, R. Gommers, P. Virtanen, D. Cournapeau, E. Wieser, J. Taylor, S. Berg, N. J. Smith, et al. Array programming with numpy. *Nature*, 585(7825):357–362, 2020. [p1]

T. Kalinowski. *quickr: Compiler for R*, 2026. URL https://github.com/t-kalinowski/quickr. R package version 0.2.1.9000, commit d38a27377e3c1b43ddaa22ba234ffdabface2454. [p2]

F. X. Konrad Krämer. *ast2ast*, 2026. URL https://github.com/Konrad1991/ast2ast/. [p3]

K. Kristensen, A. Nielsen, C. W. Berg, H. Skaug, and B. M. Bell. Tmb: Automatic differentiation and laplace approximation. *Journal of Statistical Software*, 70(5), 2016. ISSN 1548-7660. doi: 10.18637/jss.v070.i05. URL http://dx.doi.org/10.18637/jss.v070.i05. [p3]

F. Morandat, B. Hill, L. Osvald, and J. Vitek. Evaluating the design of the r language: Objects and functions for data analysis. In *European conference on object-oriented programming*, pages 104–131. Springer, 2012. [p3]

A. Munshi. The opencl specification. In *2009 IEEE Hot Chips 21 Symposium (HCS)*, pages 1–314. IEEE, 2009. [p3]

OpenXLA Team. *OpenXLA*, 2026. URL https://openxla.org/. [p1]

A. Paszke, S. Gross, F. Massa, A. Lerer, J. Bradbury, G. Chanan, T. Killeen, Z. Lin, N. Gimelshein, L. Antiga, et al. Pytorch: An imperative style, high-performance deep learning library. *Advances in neural information processing systems*, 32, 2019. [p2]

S. E. Pav. *madness: Automatic Differentiation of Multivariate Operations*, 2023. URL https://github.com/shabbychef/madness. R package version 0.2.8. [p3]

H. Perdry. *salad: Simple Automatic Differentiation*, 2024. URL https://CRAN.R-project.org/package=salad. R package version 1.2. [p3]

J. Pfeifer. *gpjrt: Go Wrappers for OpenXLA PJRT*, 2026. URL https://github.com/gomlx/gpjrt. [p1]

R Core Team. *R: A Language and Environment for Statistical Computing*. R Foundation for Statistical Computing, Vienna, Austria, 2025. URL https://www.R-project.org/. [p3]

L. Sartore. *dual: Automatic Differentiation with Dual Numbers*, 2025. URL https://CRAN.R-project.org/package=dual. R package version 0.0.6. [p3]

StableHLO Team. *StableHLO*, 2026. URL https://openxla.org/stablehlo/spec. [p2]

The IREE Authors. IREE, Sept. 2019. URL https://github.com/iree-org/iree. [p2]

A. Towell. *nabla: Exact Derivatives via Automatic Differentiation*, 2026. URL https://github.com/queelius/nabla. R package version 0.7.1. [p3]

*Sebastian Fischer*
*LMU MunichMunich Center for Machine Learning*
*Department of Statistics*
*Ludwig-Maximilians-Universität Munich*
*Ludwigstr. 33, 80539 Munich, Germany*
*ORCiD: 0000-0002-9609-3197*
sebastian.fischer@stat.uni-muenchen.de

*Daniel Falbel*
*Posit Software, PBC*

*ORCiD: 0000-0002-0912-0225*

*Thomas Kalinowski*
*Posit Software, PBC*

*Niko German*
*LMU Munich*

*ORCiD: 0009-0001-7394-8367*

*Martin Binder*
*LMU MunichMunich Center for Machine Learning*

*ORCiD: 0009-0008-2578-2869*

*Bernd Bischl*
*LMU MunichMunich Center for Machine Learning*

*ORCiD: 0000-0001-6002-6980*